

```
"""
```

```
test_loi_aire_blob.py
```

```
=====
```

```
Test n°1 du programme holographique : loi d'aire émergente.
```

```
Cadre théorique :
```

- Bekenstein-Hawking (1973-75) : $S_{\text{BH}} \propto \text{aire}(\text{horizon})$, pas volume.
- 't Hooft / Susskind / Maldacena : généralisation. Information bornée par l'aire de la frontière, pas par le volume.

```
Prédiction Blob à tester :
```

- Si système thermique/extensif standard : $\sigma^2(n_{\text{K4}} \text{ dans ball } V) \propto |V|$ (fluctuations volumétriques classiques)
- Si système holographique : $\sigma^2(n_{\text{K4}}) \propto |\partial V| \Rightarrow \propto |V|^{((d-1)/d)} = |V|^{(2/3)}$ pou.

```
Méthode :
```

- 5 seeds, équilibre MCMC canonique V109
- Pour chaque équilibre : 30 centres aléatoires, balles BFS à $r=1..4$
- Pour chaque balle : $|V|$, $|\partial V|$, $n_{\text{K4}}(V)$
- Binning logarithmique par $|V|$, mesure de $\sigma^2(n_{\text{K4}})$ par bin
- Fit log-log : exposants α (vs V) et β (vs A)

```
Diagnostics secondaires :
```

- $\langle n_{\text{K4}} \rangle$ vs V (test d'homogénéité)
- $|\partial V|$ vs $|V|$ (cross-check dimension effective)

```
"""
```

```
import numpy as np
from scipy.spatial import distance_matrix
from scipy.special import gamma as gamma_func
from scipy.stats import linregress
import time
import json
from collections import deque
```

```
# ===== Paramètres =====
```

```
KAPPA0 = 8
```

```
BETA = 0.30
```

```
TAU = 0.50
```

```
N = 200
```

```
N_STEPS = 1200
```

```
SEEDS = [0, 1, 2, 3, 4]
```

```
CENTERS_PER_SEED = 30
```

```
MAX_RADIUS = 4
```

```
# ===== Helpers Blob =====
```

```
def rips_radius(N, d, kappa0=KAPPA0):
```

```
    Vd = np.pi**(d/2) / gamma_func(d/2 + 1)
```

```
    return (kappa0 / (N * Vd))**(1/d)
```

```
def build_rips(N, d, seed):
```

```
    rng = np.random.default_rng(seed)
```

```
    pts = rng.uniform(0, 1, (N, d))
```

```
    r = rips_radius(N, d)
```

```
    dist = distance_matrix(pts, pts)
```

```
    adj = ((dist < r) & (dist > 0)).astype(np.int8)
```

```
    return pts, adj, r, dist
```

```
def count_triangles(adj):
```

```
    A = adj.astype(np.int32)
```

```
    A3 = A @ A @ A
```

```
    return int(np.trace(A3)) // 6
```

```
def count_K4_full(adj):
```

```
    """Compte K4 par arête lex-min."""
```

```
    A = adj.astype(bool)
```

```
    n = A.shape[0]
```

```
    nk4 = 0
```

```
    for i in range(n - 1):
```

```
        Ai = A[i]
```

```
        for j in range(i + 1, n):
```

```
            if not Ai[j]:
```

```
                continue
```

```
            common = Ai & A[j]
```

```
            ca = np.where(common)[0]
```

```
            ca = ca[ca > j]
```

```
            if len(ca) < 2:
```

```
                continue
```

```
            sub = A[np.ix_(ca, ca)]
```

```
            nk4 += int(np.triu(sub, 1).sum())
```

```
    return nk4
```

```
def count_K4_in_subset(adj, vertices):
```

```
    if len(vertices) < 4:
```

```
        return 0
```

```
    sub = adj[np.ix_(vertices, vertices)]
```

```
    return count_K4_full(sub)
```

```
def compute_S_sim(adj):
```

```
    deg = adj.sum(axis=1)
```

```

mu = deg.mean()
if mu < 1e-9:
    return -1e9
D = deg.std() / mu
n_tri = count_triangles(adj)
return float(np.log(1 + n_tri) - BETA * D)

def mcmc_step(adj, pool, rng, S_current):
    N = adj.shape[0]
    edges = np.argwhere(np.triu(adj, 1))
    if len(edges) == 0:
        return adj, S_current
    e_idx = rng.integers(len(edges))
    u_rm, v_rm = edges[e_idx]
    u_add = v_add = -1
    for _ in range(100):
        i, j = rng.integers(N, size=2)
        if i != j and pool[i, j] == 1 and adj[i, j] == 0:
            u_add, v_add = i, j
            break
    if u_add < 0:
        return adj, S_current
    adj_new = adj.copy()
    adj_new[u_rm, v_rm] = adj_new[v_rm, u_rm] = 0
    adj_new[u_add, v_add] = adj_new[v_add, u_add] = 1
    S_new = compute_S_sim(adj_new)
    delta = (S_new - S_current) * N / TAU
    if delta > 0 or rng.random() < np.exp(min(0.0, delta)):
        return adj_new, S_new
    return adj, S_current

def run_mcmc(N, d, seed, n_steps):
    pts, adj, r, dist = build_rips(N, d, seed)
    pool = ((dist < 1.5 * r) & (dist > 0)).astype(np.int8)
    rng = np.random.default_rng(seed + 9991)
    S_cur = compute_S_sim(adj)
    for _ in range(n_steps):
        adj, S_cur = mcmc_step(adj, pool, rng, S_cur)
    return adj

# ===== Balle géodésique BFS =====
def bfs_distances(adj, source, max_radius):
    N = adj.shape[0]
    dist = -np.ones(N, dtype=int)
    dist[source] = 0
    frontier = deque([source])
    while frontier:

```

```

        v = frontier.popleft()
        if dist[v] >= max_radius:
            continue
        neighbors = np.where(adj[v] == 1)[0]
        for u in neighbors:
            if dist[u] == -1:
                dist[u] = dist[v] + 1
                frontier.append(u)
    return dist

def measure_ball(adj, ball_vertices):
    """Retourne (|V|, |∂V|, n_K4_interior)."""
    Vsize = len(ball_vertices)
    if Vsize == 0:
        return 0, 0, 0
    in_ball = np.zeros(adj.shape[0], dtype=bool)
    in_ball[ball_vertices] = True
    A_bool = adj.astype(bool)
    boundary = 0
    for v in ball_vertices:
        outside_neighbors = A_bool[v] & ~in_ball
        boundary += int(outside_neighbors.sum())
    n_k4 = count_K4_in_subset(adj, ball_vertices)
    return Vsize, boundary, n_k4

# ===== Main =====
if __name__ == "__main__":
    t_start = time.time()
    print(f"=== Test loi d'aire (Bekenstein-Hawking analog) ===")
    print(f"N={N}, n_steps={N_STEPS}, seeds={SEEDS}")
    print(f"{CENTERS_PER_SEED} centres par seed, radii 1..{MAX_RADIUS}\n")

    records = []
    seed_summaries = []

    for seed in SEEDS:
        t0 = time.time()
        adj = run_mcmc(N, 3, seed, N_STEPS)
        n_tri = count_triangles(adj)
        n_k4_total = count_K4_full(adj)
        t_mcmc = time.time() - t0

        rng_c = np.random.default_rng(seed + 50000)
        centers = rng_c.choice(N, CENTERS_PER_SEED, replace=False)

        t1 = time.time()
        for c in centers:

```

```

    distv = bfs_distances(adj, int(c), MAX_RADIUS)
    for r in range(1, MAX_RADIUS + 1):
        ball = np.where((distv >= 0) & (distv <= r))[0]
        Vs, As, k4 = measure_ball(adj, ball)
        records.append({'seed': seed, 'center': int(c), 'r': r,
                        'V': Vs, 'A': As, 'n_K4': k4})

t_ball = time.time() - t1

seed_summaries.append({'seed': seed, 'n_tri': n_tri, 'n_K4_total': n_k4_t
                        't_mcmc': t_mcmc, 't_ball': t_ball})
print(f"Seed {seed}: n_tri={n_tri}, n_K4_total={n_k4_total}  "
      f"[MCMC {t_mcmc:.1f}s, balles {t_ball:.1f}s]")

# ----- Analyse -----
Vs_arr = np.array([r['V'] for r in records])
As_arr = np.array([r['A'] for r in records])
K4_arr = np.array([r['n_K4'] for r in records])

print(f"\nTotal points : {len(records)}")
print(f"Range |V| : [{Vs_arr.min()}, {Vs_arr.max()}] median={int(np.median(
print(f"Range |∂V| : [{As_arr.min()}, {As_arr.max()}] median={int(np.median(
print(f"Range n_K4 : [{K4_arr.min()}, {K4_arr.max()}] median={int(np.median(

# Binning logarithmique par |V|
V_lo = max(4, int(Vs_arr.min()))
V_hi = int(Vs_arr.max())
n_bins = 7
bin_edges = np.logspace(np.log10(V_lo), np.log10(V_hi + 1), n_bins + 1)

bin_stats = []
for i in range(n_bins):
    mask = (Vs_arr >= bin_edges[i]) & (Vs_arr < bin_edges[i+1])
    if mask.sum() < 4:
        continue
    bin_stats.append({
        'V_mid': float(np.mean(Vs_arr[mask])),
        'A_mid': float(np.mean(As_arr[mask])),
        'K4_mean': float(np.mean(K4_arr[mask])),
        'K4_var': float(np.var(K4_arr[mask], ddof=1)),
        'n': int(mask.sum()),
    })

print(f"\n--- Bins par |V| (n_bins utilisables = {len(bin_stats)}) ---")
print(f"{'{V}>8}  {'{A}>8}  {'{n_K4}>10}  {'σ²(n_K4)>10}  {'n:>5}")
for b in bin_stats:
    print(f"{b['V_mid']:>8.1f}  {b['A_mid']:>8.1f}  {b['K4_mean']:>10.2f}  {b

```

```

# Fits log-log
V_arr_fit = np.array([b['V_mid'] for b in bin_stats])
A_arr_fit = np.array([b['A_mid'] for b in bin_stats])
K4mean_fit = np.array([b['K4_mean'] for b in bin_stats])
sig2_fit = np.array([b['K4_var'] for b in bin_stats])

print(f"\n--- Fits log-log ---")
fits = {}

# (n_K4) vs V (test d'homogénéité – devrait être ~1 si Blob homogène)
mask = (V_arr_fit > 0) & (K4mean_fit > 0)
if mask.sum() >= 3:
    res = linregress(np.log(V_arr_fit[mask]), np.log(K4mean_fit[mask]))
    fits['mean_K4_vs_V'] = {'exp': float(res.slope), 'r2': float(res.rvalue**2),
                           'intercept': float(res.intercept)}
    print(f"{n_K4} ∝ V^α      : α = {res.slope:.3f} (R²={res.rvalue**2:.3f})")
    print(f"expected ≈ 1.0 si K4 distribués uniformément")

# |∂V| vs |V|
mask = (V_arr_fit > 0) & (A_arr_fit > 0)
if mask.sum() >= 3:
    res = linregress(np.log(V_arr_fit[mask]), np.log(A_arr_fit[mask]))
    fits['A_vs_V'] = {'exp': float(res.slope), 'r2': float(res.rvalue**2)}
    d_eff = 1.0 / (1.0 - res.slope) if res.slope < 1 else float('inf')
    print(f"|∂V| ∝ V^γ      : γ = {res.slope:.3f} (R²={res.rvalue**2:.3f})")
    print(f"d_eff = 1/(1-γ) = {d_eff:.2f} (théorique d=3 ⇒")

# σ²(n_K4) vs V – LE TEST CENTRAL
mask = (V_arr_fit > 0) & (sig2_fit > 0)
if mask.sum() >= 3:
    res = linregress(np.log(V_arr_fit[mask]), np.log(sig2_fit[mask]))
    fits['sigma2_vs_V'] = {'exp': float(res.slope), 'r2': float(res.rvalue**2)}
    print(f"σ²(n_K4) ∝ V^α : α = {res.slope:.3f} (R²={res.rvalue**2:.3f})")
    print(f"α ≈ 1.0 ⇒ extensif/thermique (poissonien)")
    print(f"α ≈ 0.667 ⇒ loi d'aire (Bekenstein-Hawking d=3)")
    print(f"α ≈ 1.33 ⇒ super-extensif (corrélations longue")

# σ²(n_K4) vs A
mask = (A_arr_fit > 0) & (sig2_fit > 0)
if mask.sum() >= 3:
    res = linregress(np.log(A_arr_fit[mask]), np.log(sig2_fit[mask]))
    fits['sigma2_vs_A'] = {'exp': float(res.slope), 'r2': float(res.rvalue**2)}
    print(f"σ²(n_K4) ∝ A^β : β = {res.slope:.3f} (R²={res.rvalue**2:.3f})")
    print(f"β ≈ 1.0 ⇒ confirmation loi d'aire directe")

# ----- Verdict -----
print(f"\n--- VERDICT ---")

```

```

alpha = fits.get('sigma2_vs_V', {}).get('exp', None)
if alpha is not None:
    if alpha < 0.78:
        print(f" $\sigma^2(n_{K4}) \propto V^{\{\alpha:.2f\}}$  : signature compatible loi d'aire ho
        print(f" ( $\alpha < 1$  significatif, proche du 2/3 attendu)")
        print(f"  $\Rightarrow$  [I] indication forte. Mérite scale-up Colab.")
    elif alpha > 1.2:
        print(f" $\sigma^2(n_{K4}) \propto V^{\{\alpha:.2f\}}$  : super-extensif (corrélations longu
        print(f"  $\Rightarrow$  phénomène intéressant mais ce n'est PAS holographique au
    else:
        print(f" $\sigma^2(n_{K4}) \propto V^{\{\alpha:.2f\}}$  : régime extensif/thermique standard
        print(f"  $\Rightarrow$  pas de loi d'aire détectée. [FAIL] honnête pour holograp

t_total = time.time() - t_start
print(f"\nTemps total : {t_total:.1f}s")

# Sauvegarde
out = {
    'params': {'N': N, 'n_steps': N_STEPS, 'seeds': SEEDS,
               'centers_per_seed': CENTERS_PER_SEED, 'max_radius': MAX_RADIUS
               'kappa0': KAPPA0, 'beta': BETA, 'tau': TAU},
    'seed_summaries': seed_summaries,
    'bins': bin_stats,
    'fits': fits,
    'wall_time_s': t_total,
    'n_records': len(records),
}
with open('/home/claude/loi_aire_results.json', 'w') as f:
    json.dump(out, f, indent=2)
print(f"Résultats : /home/claude/loi_aire_results.json")

```